

ISIMM | 2025

DÉVELOPPEMENT DES APPLICATIONS RÉPARTIES

Audience: LI3

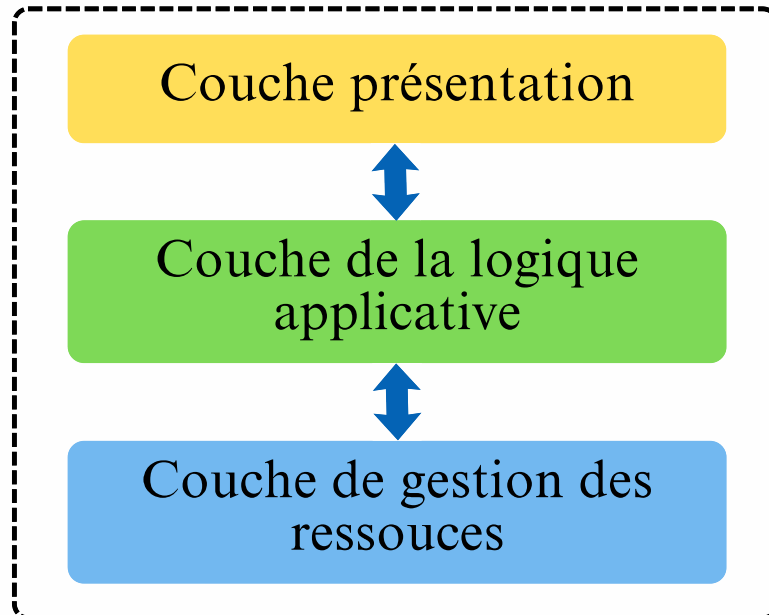
Enseignant: Dr. Marwa Thabet
Maître assistant @ISIMM

Réflexions & attentes ?



1. Les systèmes d'information

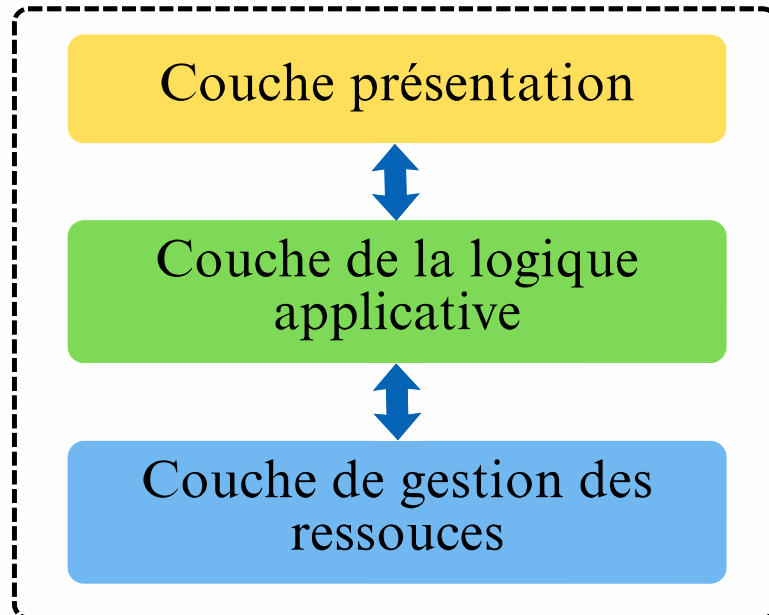
• Organisation



- La **couche de présentation** a pour rôle de présenter les informations aux entités externes (utilisateurs humains ou autres ordinateurs) et de leur offrir un moyen d'interagir avec le système, en soumettant des opérations et en recevant des réponses.

1. Les systèmes d'information

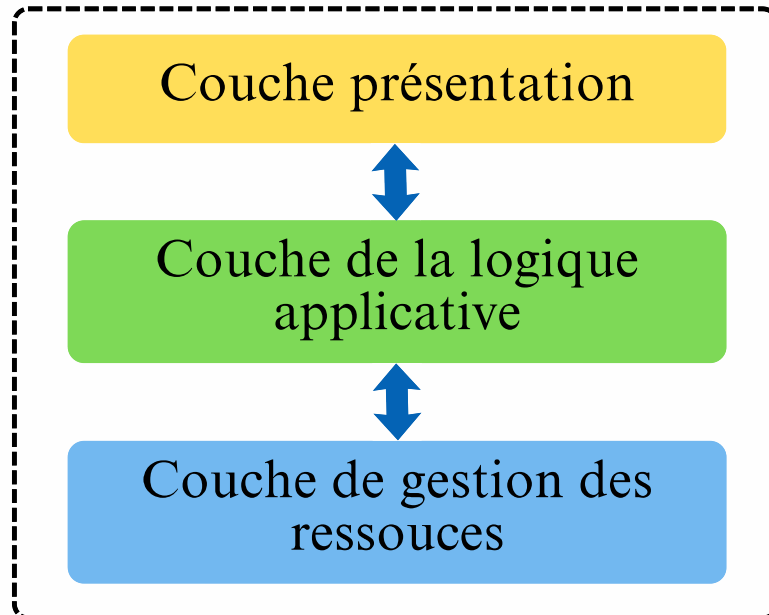
• Organisation



- La **logique applicative** veille à exécuter/mener les différentes tâches/opérations du système. Elle peut être: programme, contraintes, règles, processus métiers.

1. Les systèmes d'information

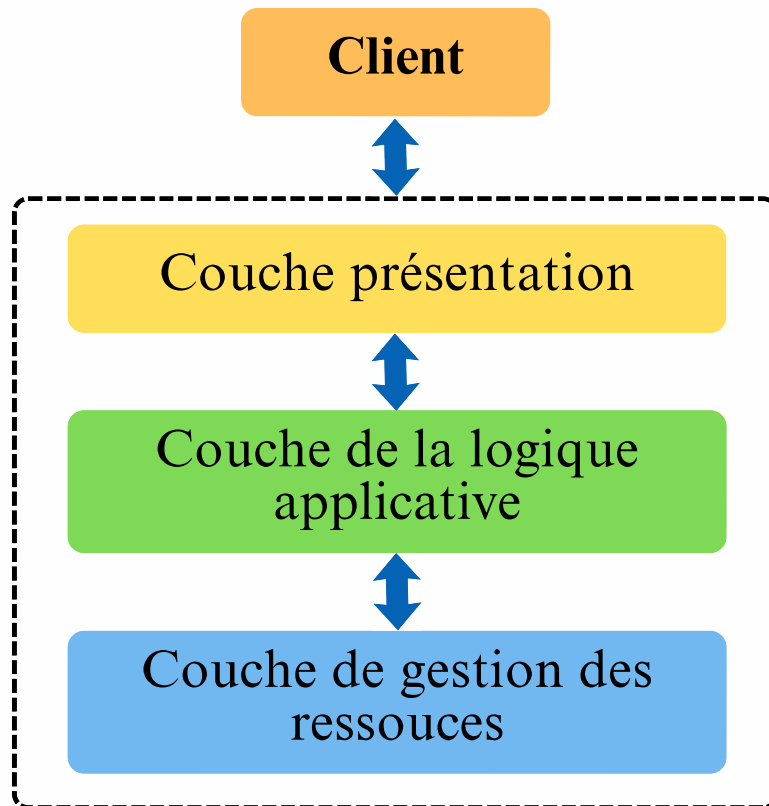
• Organisation



- Le **gestionnaire de ressource** est responsable sur le stockage, l'indexation et l'interrogation des donnés. ça peut être tout système de gestion de données permettant la persistance et l'interrogation des données: une base de données, système de gestion de données textuelles.

1. Les systèmes d'information

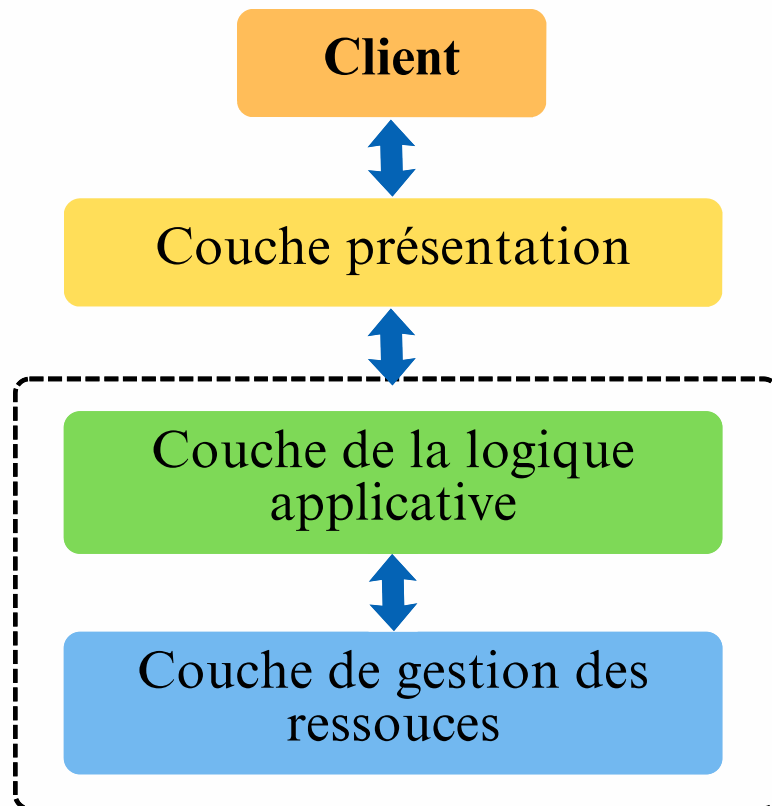
• Organisation



- Un **système monolithique (1-tier)** qui intègre les 3 couches.
- Les utilisateurs/programmes accèdent au système via des terminaux (simple affichage).
- C'est l'architecture des premiers systèmes mainframes.
- **Avantages:**
 - **Pas de changement de contexte** : le flot de contrôle reste linéaire et simple, ce qui rend l'exécution rapide.
 - **Gestion des ressources centralisée** : facile à contrôler et à administrer puisque tout est regroupé dans une seule machine.
 - **Conception optimisée** : comme il n'y a qu'un seul bloc logiciel, on peut optimiser finement ses performances.

1. Les systèmes d'information

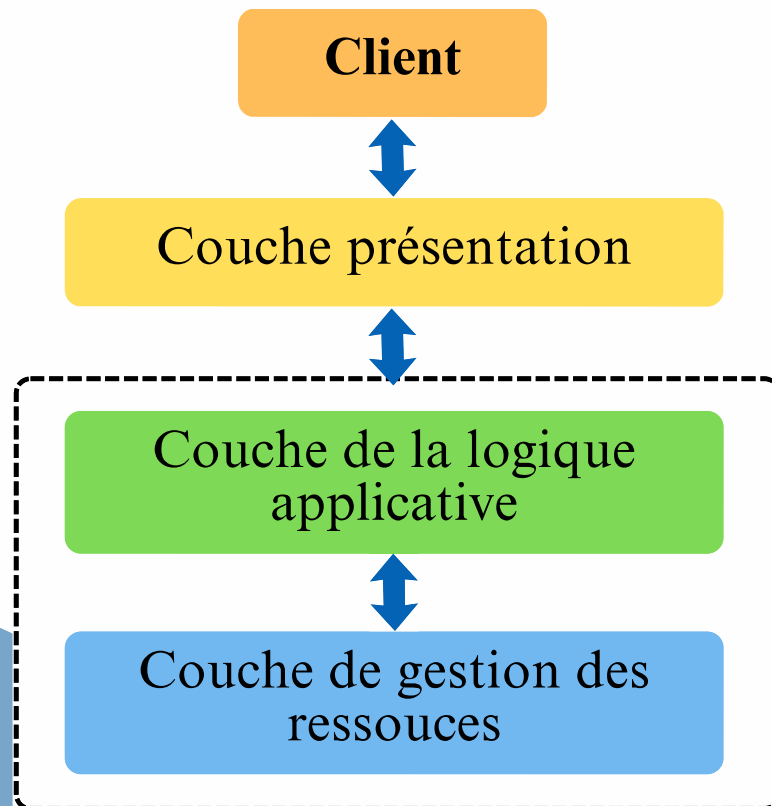
• Organisation



- Un système **client/serveur (2-tiers)** qui déplace la couche de présentation au client.
 - Les utilisateurs/programmes accèdent au système via des terminaux (simple affichage).
 - C'est l'architecture des premiers systèmes mainframes.
 - **Avantages:**
 - Les clients sont indépendants.
 - Prend en compte les capacités de calcul des clients.
 - Introduit le concept de API (Application Program Interface). Une interface pour invoquer le système de l'extérieur.
 - Le gestionnaire de ressource voit seulement un seul client: une seule couche applicative.
- ➔ **Bonne performance puisque il y a pas de connexions/session des clients à maintenir.**

1. Les systèmes d'information

• Organisation



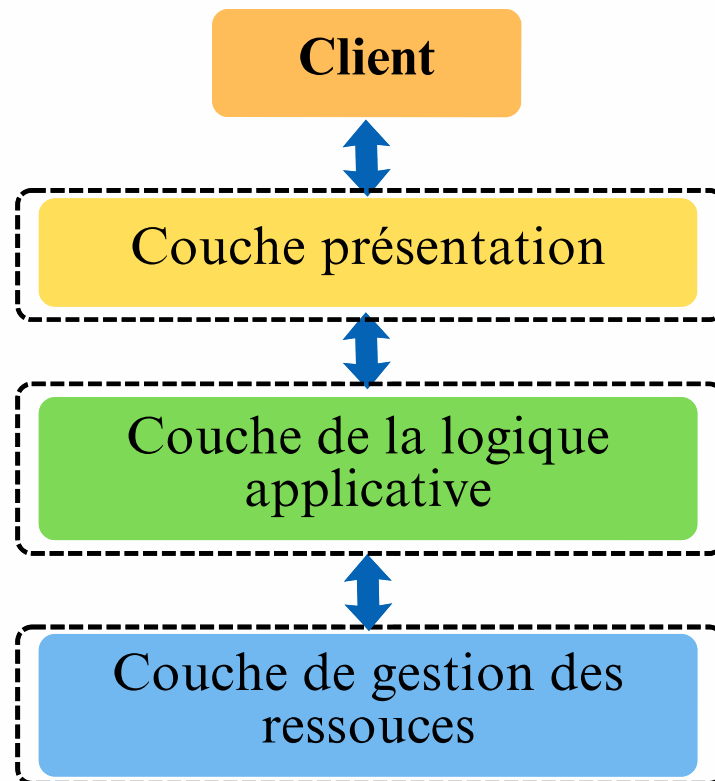
• Limites:

- Si un client veut accéder à 2 serveurs ou plus, l'architecture 2 tiers présente les problèmes suivants:
 - Les systèmes sous-jacents ne se connaissent pas
 - Il n'y a pas une logique métier commune
- Le client est le point d'intégration (client de plus en plus lourd)
- La responsabilité de gérer l'hétérogénéité des systèmes est transmise aux clients.
- Le client devient responsable de savoir les localisations des entités, comment les utiliser et comment assurer la cohérence.
- Ceci est inefficace de plusieurs points de vue (conception logicielle, portabilité et réutilisation de code, performance).

➡ Ces problèmes sont inhérents à l'architecture 2-tiers.

1. Les systèmes d'information

• Organisation



Pour une architecture 3-tiers:

- Les trois couches sont complètement séparées.
- Typiquement, les couches sont aussi distribuées.
- Modularité de la conception
- Logique applicative pas nécessairement centralisée.
- Les architectures 3-tiers font généralement intervenir une couche intergiciel (middleware).
- Les même problèmes que dans les systèmes 2-tiers.

Chacun de ces tiers n'est pas seulement un logiciel, mais il s'exécute réellement comme un **processus** sur une machine.

2. Rappel sur les processus

Un processus

- **Définition:** entité dynamique représentant l'exécution d'un programme sur une machine.

- **Caractéristiques:**

- Dispose de ressources allouées (CPU, mémoire, périphériques).
- Possède un état d'exécution (données en mémoire, registres, pile).

- **Exécution:**

- Sur une machine multi-CPU, plusieurs processus peuvent s'exécuter en parallèle.
- Sur une machine mono-CPU, ils s'exécutent en concurrence (ordonnancement par le système).

2. Rappel sur les processus Parallélisme

- **Définition:** exécution simultanée de plusieurs processus (ou activités) sur plusieurs processeurs.
- **Condition :** possible uniquement sur une architecture multi-CPU.
- **Exemple :**
 - Une mémoire partagée.
 - Un ensemble de processeurs exécutant différents processus en même temps.

2. Rappel sur les processus

Exécution concurrente de processus

- **Systèmes concurrents:** deux ou plusieurs processus (activités) coopèrent pour accomplir une tâche. La coopération possible via :

- Variables partagées
- Échange de messages

Processeur P1		Processeur P2
(1) $x \leftarrow 1$		(a) $y \leftarrow 1$
(2) $t1 \leftarrow y$		(b) $t2 \leftarrow x$

2. Rappel sur les processus

Exécution concurrente de processus

- Concurrency:

- Abstraction du parallélisme
- Exécution alternée (entrelacement) des processus sur un ou plusieurs processeurs.
- $\neq$ Parallélisme : les processus ne s'exécutent pas toujours en même temps, mais donnent l'illusion du simultané.

Processeur P1

(1) $x \leftarrow 1$
(2) $t1 \leftarrow y$

Processeur P2

(a) $y \leftarrow 1$
(b) $t2 \leftarrow x$

2. Rappel sur les processus

Exécution concurrente de processus

Processeur P1	Processeur P2
(1) $x \leftarrow 1$	(a) $y \leftarrow 1$
(2) $t1 \leftarrow y$	(b) $t2 \leftarrow x$

- **Instruction atomique** : exécution indivisible, effectuée sans interruption.
- **Processus séquentiel** : enchaînement ordonné d'instructions atomiques.
- **Système concurrent** : composé de plusieurs processus séquentiels s'exécutant de façon potentiellement entrelacée.

2. Rappel sur les processus

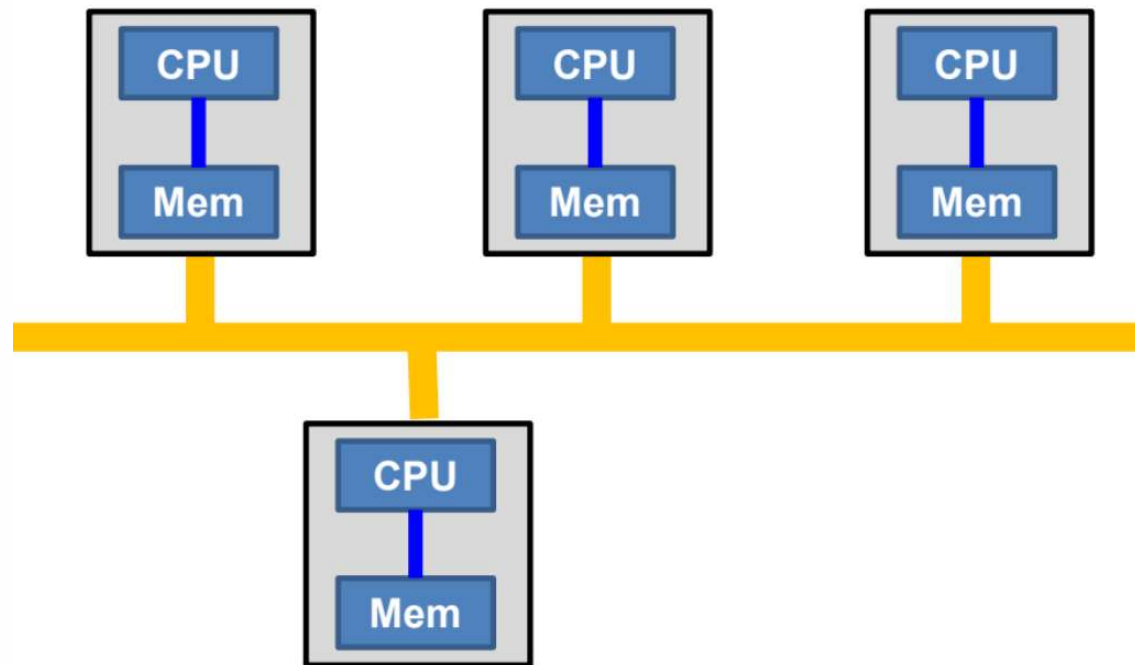
Exécution concurrente de processus

Modèle de l'entrelacement

- On raisonne sur des exécutions séquentielles obtenues par l'entrelacement des instructions des différentes activités.
- Exécution d'un système concurrent = entrelacement arbitraire des instructions atomiques des processus séquentiels.
- Cet entrelacement doit toujours respecter l'ordre interne de chaque processus.

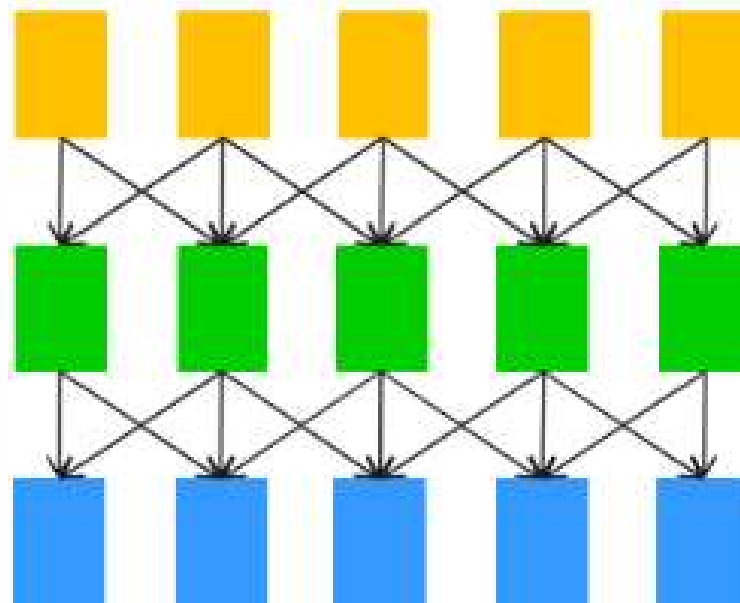
Deux activités $P = p_1; p_2$ et $Q = q_1; q_2$. L'exécution concurrente $P \parallel Q$ est vue comme (équivalente à) l'une des exécutions :
 $p_1; p_2; q_1; q_2$ ou $p_1; q_1; p_2; q_2$ ou $p_1; q_1; q_2; p_2$ ou $q_1; p_1; p_2; q_2$ ou
 $q_1; p_1; q_2; p_2$ ou $q_1; q_2; p_1; p_2$

- Un **système distribué** est un ensemble de processus exécutés sur des machines distinctes, qui communiquent entre eux via un réseau et coopèrent afin d'atteindre un objectif commun.



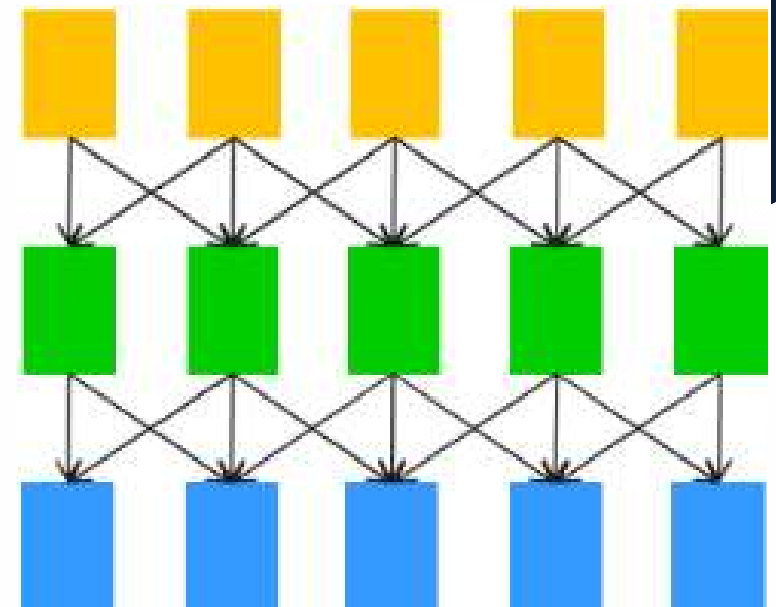
Interactions dans les SID

- Comment connecter les différentes couches et modules d'un SID?
- Comment les différentes parties du système interagissent à travers des interactions entre les modules logicielles?



Communication inter processus (IPC)

- Permet la communication entre des processus
- Transférer des données
- Invoquer une action dans un autre processus (éventuellement s'exécutant dans une autre machine distante).



Contenu du module

1. Programmation réseaux avec les sockets
2. Applications Client/ Serveur
3. Intergiciels orientés objets/ Intergiciels orientés messages
4. Problèmes fondamentaux de la répartition



Chapitre 1

Programmation réseaux avec les sockets



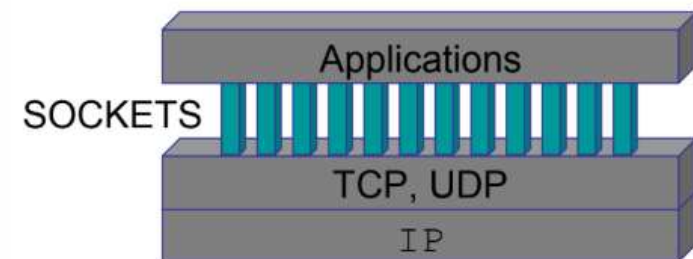
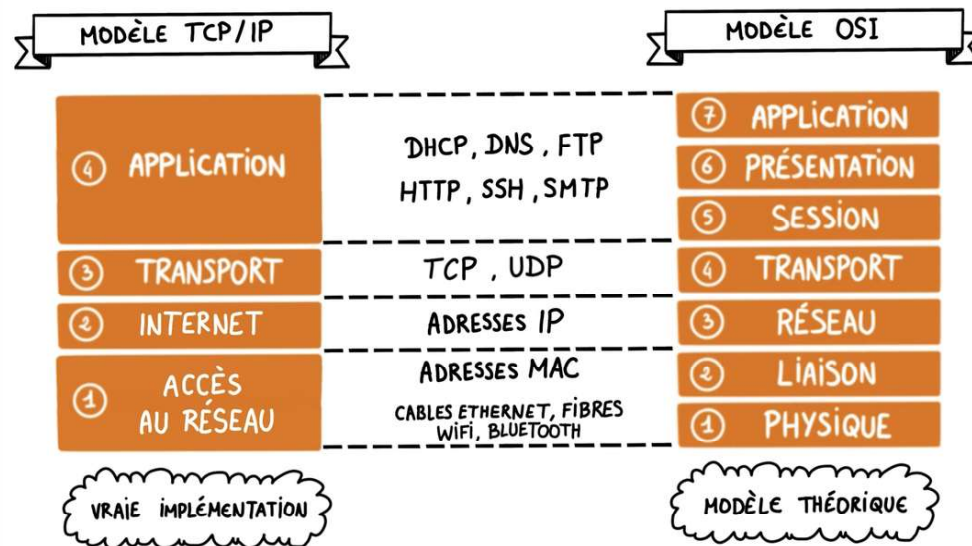
Plan

1. Concept de socket
2. Socket TCP
3. Socket UDP
4. API Java.net.*



1. Concept de socket

- La **notion de sockets** a été introduite dans les distributions Berkeley UNIX (BSD), d'où l'appellation courante de sockets BSD.
- Elles se situent juste au-dessus de la couche transport du modèle OSI (protocoles TCP ou UDP), laquelle repose elle-même sur les services de la couche réseau (protocole IP).



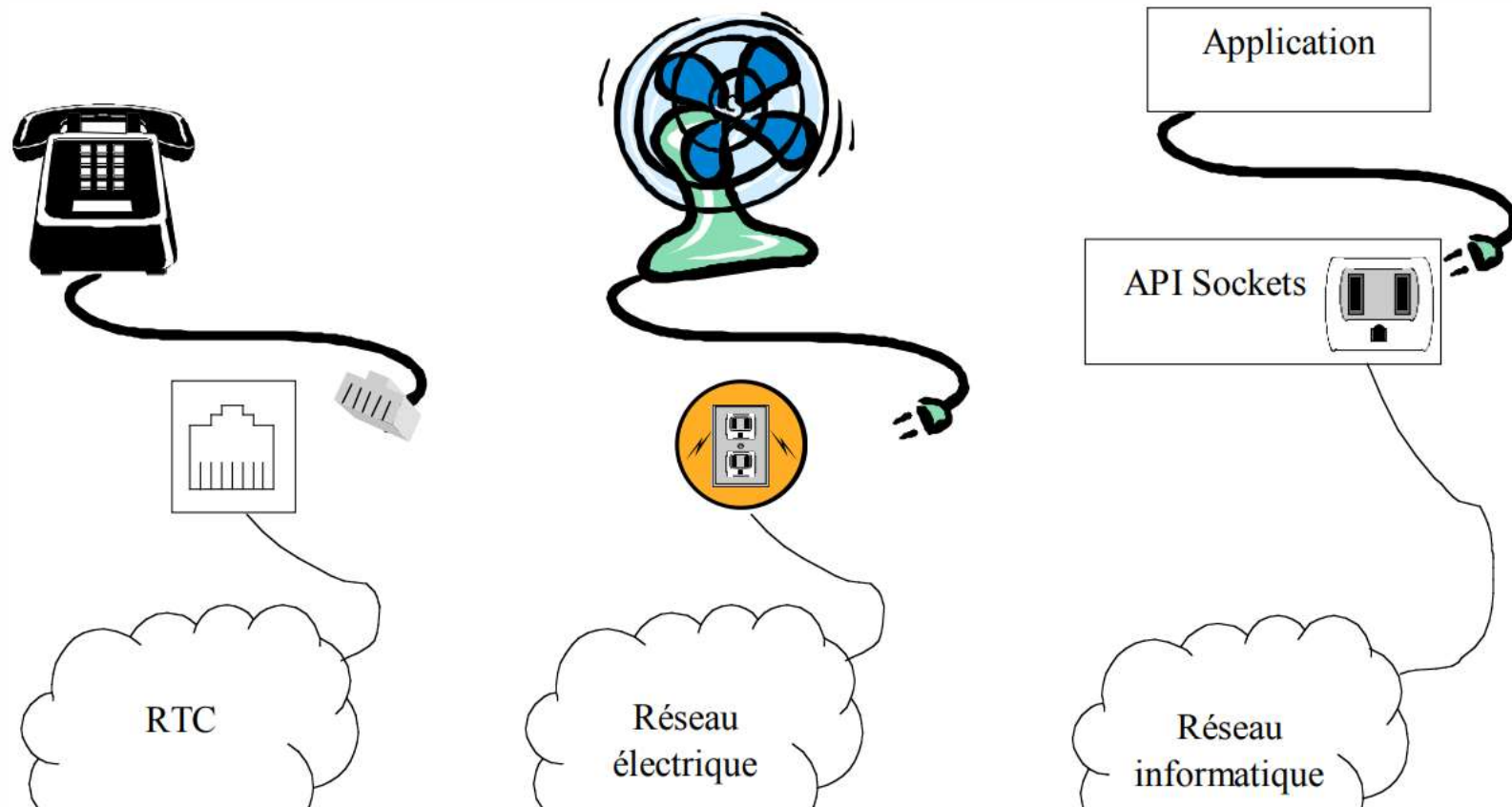
1. Concept de socket

- Le protocole IP est aujourd'hui le plus utilisé pour la communication entre machines. Toutefois, il fournit un service d'envoi de paquets non fiable (sans garantie de livraison ni d'ordre). Il est donc généralement complété par des protocoles de transport :
 - TCP : offre un service de communication fiable, orienté connexion, avec contrôle de flux et correction d'erreurs.
 - UDP : fournit un service non fiable basé sur l'envoi de datagrammes, mais plus léger et plus rapide que TCP.

1. Concept de socket

- Les protocoles TCP/IP et UDP/IP sont accessibles aux applications au moyen des sockets.
- Un socket joue le rôle de point d'accès entre une application et les services fournis par les couches TCP et UDP.
- Ce concept est similaire à l'abstraction de fichier dans UNIX : de la même manière qu'un fichier permet à un programme d'accéder au système de stockage, un socket permet à une application d'accéder au réseau.

1. Concept de socket



1. Concept de socket

- Les sockets offrent :
 - Un mécanisme de communication bidirectionnel interprocessus dans un environnement distribué.
 - Une API (Application Programming Interface) permettant aux programmeurs d'interfacer leurs applications avec les protocoles de communication sous-jacents.
- **Remarque:** L'interface des sockets est indépendante de toute pile de protocoles particulière. Dans le cadre de ce cours, nous nous focaliserons sur leur utilisation avec les protocoles TCP/IP et UDP/IP.

1. Concept de socket

En fonction du protocole de transport utilisé, les sockets adoptent un mécanisme de communication différent.

Mode connecté (TCP)

- Comparable à un appel téléphonique
- Connexion durable entre les processus
- Communication fiable, orientée flux
- Basé sur le modèle client/serveur

Mode non connecté (UDP)

- Comparable à un envoi de courrier
- Adresse de destination requise à chaque envoi
- Pas d'accusé de réception
- Communication par datagrammes indépendants
- Supporte la multidiffusion (communication vers un groupe de machines)

2. Sockets TCP

Le modèle client/serveur (TCP)

- TCP offre un service connecté et fiable, avec livraison des données dans l'ordre.
- La connexion est asymétrique à l'établissement :
 - Serveur : attend les demandes de connexion.
 - Client : initie la connexion.
- Après connexion, le fonctionnement devient symétrique.
- Deux types de sockets côté serveur :
 - Socket d'écoute : reçoit les demandes de connexion.
 - Socket de service : communication avec un client (un par client connecté).
- Un serveur peut être connecté simultanément à plusieurs clients.

2. Sockets TCP

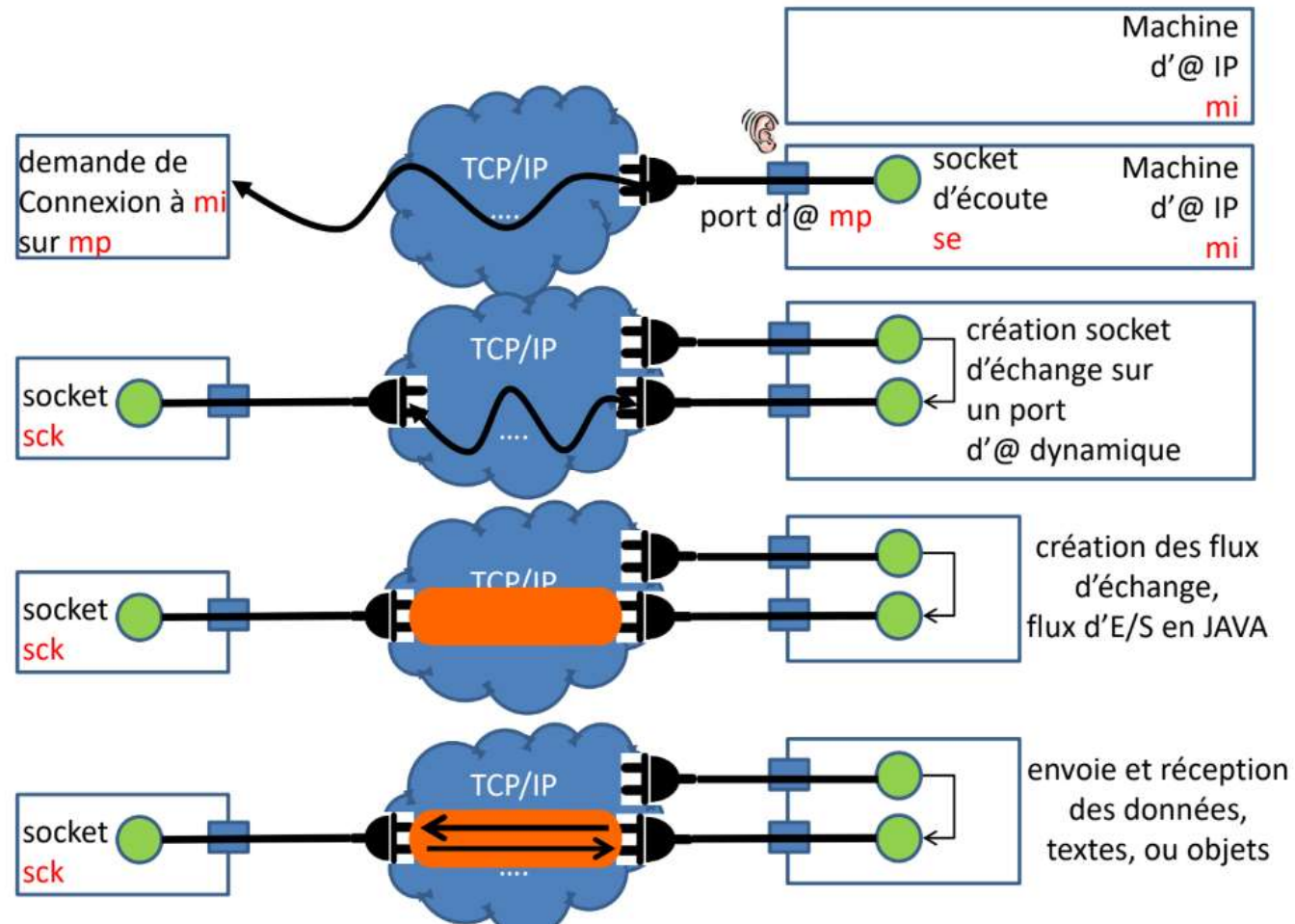
Client

- 3 Demande de connexion sur la machine d'adresse **mi** et le port **mp** avec le socket d'échange de flux **sck**
- 5 création des flux pour échanger des octets, de texte, ou d'objets via **sck**
- 6 échange via les flux créés
- 7 fermer le socket **sck**

• Serveur

- 1 Créer un socket d'écoute **se** sur la machine d'adresse IP **mi** et le port **mp**
- 2 se mettre en attente / écoute de demande de connexions sur **se**
- 4 réception d'une demande de connexion
→ création de socket d'échange **sckServ**
- 5 création des flux pour échanger des octets, de texte, ou d'objets via **sckServ**
- 6 échange via les flux créés
- 7 fermer le socket **sckServ**
- 8 fermer le socket **se**

2. Sockets TCP



2. Sockets TCP

Création de socket

• Côté serveur

/* Créer un socket d'écoute se sur la machine
d'adresse IP **mi** et le port **mp** */

```
ServerSocket se = new ServerSocket(mp);
```

/* se mettre en attente / écoute de demande de
connexions sur se */

```
Socket sckServ = se.accept();
```

• Côté client

/* Demande de connexion sur la machine
d'adresse **mi** et le port **mp** avec le socket
d'échange de flux sck */

```
Socket sck = new Socket (mi , mp);
```

2. Sockets TCP

Flux d'échange

- Etant donné un socket **sck**

```
InputStream is = sck.getInputStream();  
int nb = is.read();
```

```
OutputStream os= sck.getOutputStream();  
int res = nb*3;  
os.write(res);
```

2. Sockets TCP

Flux d'échange

- Etant donné un socket **sck**

```
InputStream is = sck.getInputStream();
```

```
InputStreamReader isr = new InputStreamReader(is);
```

```
BufferedReader br = new BufferedReader(isr);
```

```
String str=br.readLine();
```

```
OutputStream os= sck.getOutputStream();
```

```
PrintWriter pw = new PrintWriter(os,true);
```

```
String rep="Bonjour Mr"+str;
```

```
pw.println(rep); //os.write(res);
```

2. Sockets TCP

Flux d'échange

- Échange d'objets en Java : **Sérialisation / Désérialisation**
 - ♦ **Sérialisation**: Transformation d'un objet (en mémoire) en une suite d'octets transmissibles sur le réseau.
 - ♦ **Désérialisation**: Processus inverse : octets reçus → recréation de l'objet original.
 - ♦ Condition : l'objet doit être sérialisable
 - ♦ La classe doit implémenter l'interface **Serializable**.

2. Sockets TCP

Flux d'échange

- Etant donné un socket **sck**

```
OutputStream os = sck.getOutputStream();  
ObjectOutputStream oos = new ObjectOutputStream(os);  
Compte cpt = new Compte("1200", 1000);  
oos.writeObject(cpt);
```

```
InputStream is = sck.getInputStream();  
ObjectInputStream ois = new ObjectInputStream(is);  
Compte cptRecu = (Compte) ois.readObject();
```

2. Sockets TCP

Exemple 1 de communication via des sockets

/* Serveur mono-client */

```
import java.net.*;
import java.io.*;

public class Serveur1 {
    public static void main(String[] args){
        try {
            ServerSocket ss = new ServerSocket(1234);
            System.out.println("J'attends une connexion");
            Socket s = ss.accept();
            InputStream is=s.getInputStream();
            OutputStream os=s.getOutputStream();
            System.out.println("J'attends un nombre");
            int nb=is.read();
            int res=nb*3;
            System.out.println("J'envoies la réponse");
            os.write(res);
            s.close();
        } catch (Exception e){ e.printStackTrace(); } }
```

/*Client */

```
import java.net.*; import java.io.*;
import java.util.Scanner;

public class Client1 {
    public static void main(String[] args){
        try {
            Socket s = new Socket("localhost", 1234);
            InputStream is=s.getInputStream();
            OutputStream os=s.getOutputStream();
            Scanner clavier = new Scanner(System.in);
            System.out.println("Donner un nombre");
            int nb=clavier.nextInt();
            os.write(nb);
            int res = is.read();
            System.out.println("Res =" +res);
            os.write(res);
            s.close();
        } catch (Exception e){ e.printStackTrace(); } }
```

2. Sockets TCP

Exemple 2 de communication via des sockets

/* Serveur mono-client */

```
import java.net.*; import java.io.*;

public class Serveur2 {

    public static void main(String[] args){

        try {ServerSocket ss = new ServerSocket(1234);

        System.out.println("J'attends une connexion");

        Socket s = ss.accept();

        InputStream is=s.getInputStream();

        InputStreamReader isr = new InputStreamReader(is);

        BufferedReader br = new BufferedReader(isr);

        OutputStream os=s.getOutputStream();

        PrintWriter pw = new PrintWriter(os,true);

        System.out.println("J'attends une chaine");

        String str=br.readLine(); //int nb=is.read();

        String rep="Bonjour Mr"+str;

        System.out.println("J'envoies la réponse");

        pw.println(rep); //os.write(res);

        s.close();

    } catch (Exception e){ e.printStackTrace(); } }
```

/*Client */

```
import java.net.*;

import java.io.*; import java.util.Scanner;

public class Client2 {

    public static void main(String[] args){

        try { Socket s = new Socket("localhost", 1234);

        InputStream is=s.getInputStream();

        InputStreamReader isr = new InputStreamReader(is);

        BufferedReader br = new BufferedReader(isr);

        OutputStream os=s.getOutputStream();

        PrintWriter pw = new PrintWriter(os,true);

        Scanner clavier = new Scanner(System.in);

        System.out.println("Donner votre nom");

        String str=clavier.next();

        pw.println(str);

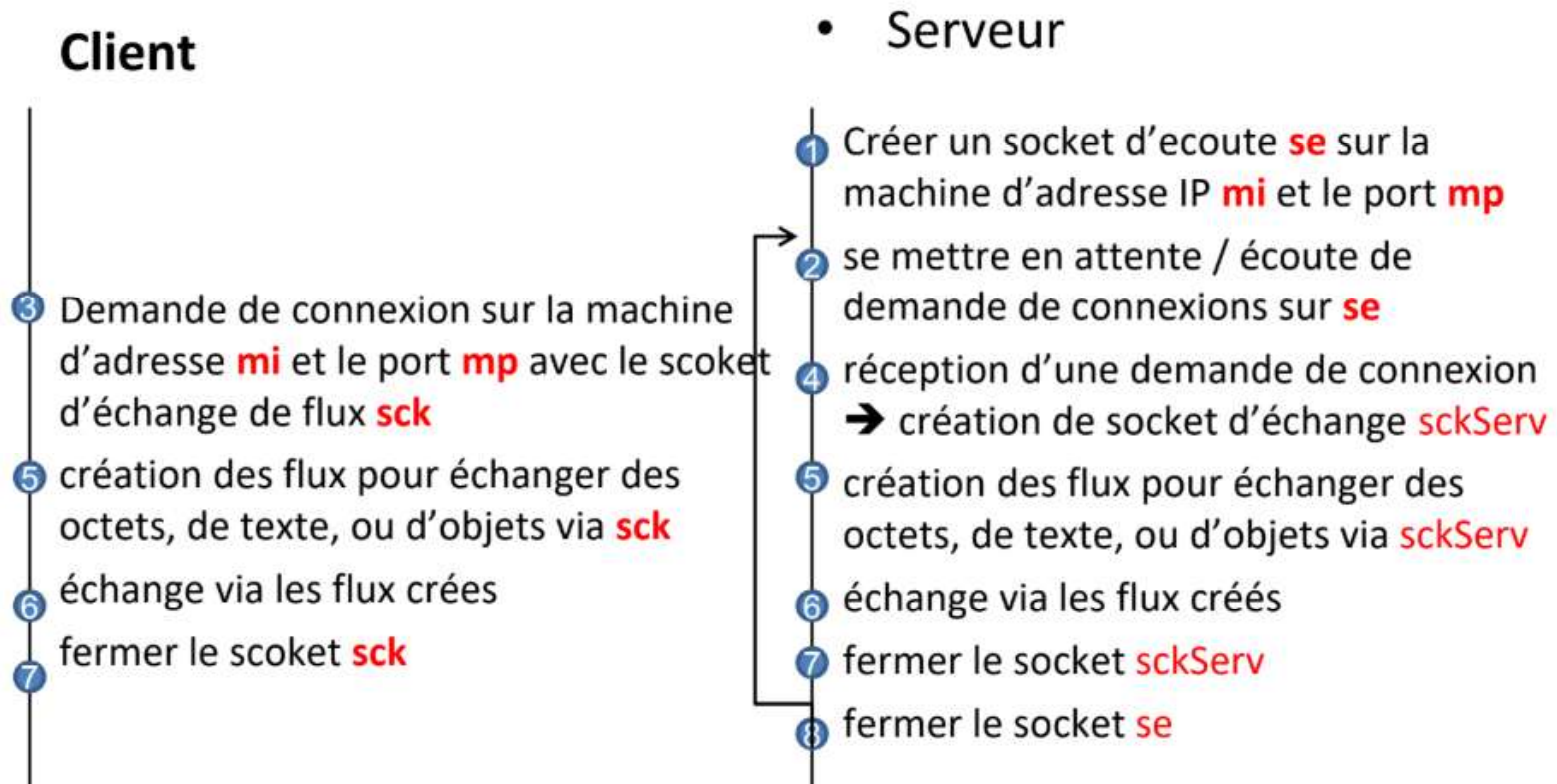
        String res = br.readLine();

        System.out.println("Res =" +res);

    } catch (Exception e){ e.printStackTrace(); } }
```

2. Sockets TCP

Exemple 3 de communication via des sockets –
Serveur multi-clients itératifs



2. Sockets TCP

Exemple 3 de communication via des sockets – Serveur multi-clients itératifs

```
import java.io.*;

public class Serveur3 {
    public static void main(String[] args){
        try { ServerSocket ss = new ServerSocket(123);
            while(true){
                System.out.println("J'attends une connexion");
                Socket s = ss.accept();
                InputStream is=s.getInputStream();
                InputStreamReader isr = new
                InputStreamReader(is);
                BufferedReader br = new BufferedReader(isr);
```

```
                OutputStream os=s.getOutputStream();
                PrintWriter pw = new PrintWriter(os,true);
                System.out.println("J'attends une chaine");
                String str=br.readLine();
                String rep="Bonjour Mr"+str;
                System.out.println("J'envoies la réponse");
                pw.println(rep);
                s.close();
            } } catch (Exception e){ e.printStackTrace(); }
        } }
```

2. Sockets TCP

Exemple 4 de communication via des sockets – Serveur multi-threads

♦ Problème :

Un serveur classique (mono-thread) ne peut traiter qu'un seul client à la fois → les autres doivent attendre.

♦ Solution :

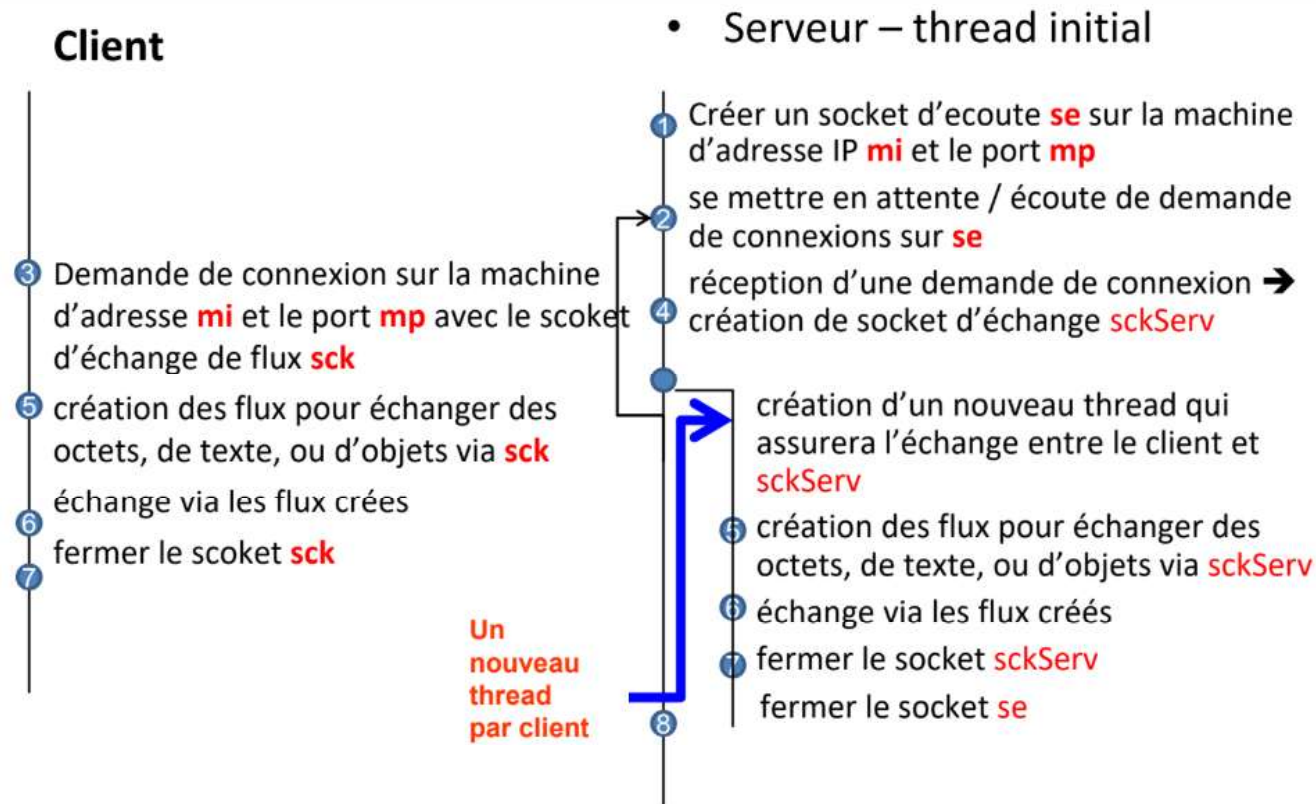
Utiliser le multi-threading pour gérer plusieurs clients simultanément.

♦ Principe

1. Le serveur principal reste en attente (accept()) de nouvelles connexions.
2. À chaque nouvelle connexion :
 - Un nouveau thread est créé.
 - Ce thread est associé à la socket du client.
3. Ce thread dédié gère les opérations d'entrée/sortie (read/write) avec le client.
4. Le serveur principal continue à accepter d'autres connexions.

2. Sockets TCP

Exemple 4 de communication via des sockets – Serveur multi-threads



2. Sockets TCP

Exemple 4 de communication via des sockets – Serveur multi-threads

```
import java.net.*; import java.io.*;

public class ServeurMT extends Thread{

    @Override
    public void run(){
        try {

            ServerSocket ss = new ServerSocket(123);

            while(true){

                System.out.println("J'attends une connexion");

                Socket s = ss.accept();

                new Service(s,nbClients).start();

            }

        } catch (Exception e){ e.printStackTrace(); }
    }

    public static void main(String[] args){
        new ServeurMT().start();
    }
}
```

2. Sockets TCP

Exemple 4 de communication via des sockets – Serveur multi-threads

```
import java.net.*; import java.io.*;

public class Service extends Thread {

    private Socket s;

    private int numClient;

    public Service(Socket socket, int numClient){
        super(); this.s = socket; this.numClient = numClient; }

    public void run(){
        try{
            InputStream is=s.getInputStream();
            InputStreamReader isr = new InputStreamReader(is);

            BufferedReader br = new BufferedReader(isr);

            OutputStream os=s.getOutputStream();

            PrintWriter pw = new PrintWriter(os,true);

            pw.println("Voues êtes client numero"+numClient);

            System.out.println("Connexion du client Num "+ numClient);

            System.out.println("IP = "+s.getRemoteSocketAddress());

            while(true){ String req=br.readLine(); pw.println(req.length());
                }

            } catch (Exception e){ e.printStackTrace(); }

        }
    }
```

2. Sockets TCP

Exemple 4 de communication via des sockets – Serveur multi-threads

